

TD Chapitre 1 : Représentation Numérique

Conversions, Entiers et Flottants

Dr. El Hadji Bassirou TOURÉ
Département de Mathématiques et Informatique
Université Cheikh Anta Diop de Dakar

1 Conversions de Base

Exercice 1 : Conversions binaire $\leftrightarrow$ hexadécimal $\leftrightarrow$ décimal

Compléter le tableau suivant (valeurs non signées sur 8 bits) :

Binaire (8 bits)	Hexadécimal	Décimal
10110101		
	0x7F	
		200
11001010		
	0xE3	
		47

Exercice 2 : Conversion décimal $\rightarrow$ binaire par divisions successives

En utilisant la méthode des divisions successives par 2, convertir en binaire sur 8 bits :

- a) 73_{10}
- b) 156_{10}
- c) 241_{10}

Pour chaque conversion, montrer toutes les étapes (divisions et restes).

Exercice 3 : Conversions rapides hex $\leftrightarrow$ binaire

Convertir directement (sans passer par le décimal) :

- a) $0xAF$ en binaire
- b) $0x3C$ en binaire
- c) 11110001_2 en hexadécimal
- d) 01011110_2 en hexadécimal

Rappel : chaque chiffre hex correspond exactement à 4 bits.

2 Entiers Signés et Non Signés

Exercice 4 : Formule B2U (Binary to Unsigned)

En utilisant la formule $B2U_w(\vec{x}) = \sum_{i=0}^{w-1} x_i \cdot 2^i$, calculer la valeur décimale des motifs binaires suivants interprétés comme **unsigned** sur 8 bits :

- a) 10000001_2
- b) 11111111_2
- c) 01010101_2

Exercice 5 : Formule B2T (Binary to Two's complement)

En utilisant la formule $B2T_w(\vec{x}) = -x_{w-1} \cdot 2^{w-1} + \sum_{i=0}^{w-2} x_i \cdot 2^i$, calculer la valeur décimale des mêmes motifs interprétés comme **signed** sur 8 bits :

- a) 10000001_2
- b) 11111111_2
- c) 01010101_2

Exercice 6 : Négation en complément à deux

Calculer la négation des nombres suivants sur 8 bits en utilisant $-x = \sim x + 1$:

- a) $x = 42$ (écrire d'abord en binaire)
- b) $x = 1$
- c) $x = -1$ (soit 11111111_2)
- d) $x = -128$ (soit 10000000_2) — que remarquez-vous ?

Pour chaque cas, montrer : x en binaire, $\sim x$, puis $\sim x + 1$.

Exercice 7 : Plages de valeurs

- a) Donner T_{min} et T_{max} pour un entier signé sur 8 bits.
- b) Donner U_{min} et U_{max} pour un entier non signé sur 8 bits.
- c) Donner T_{min} et T_{max} pour `int16_t` (16 bits signés).
- d) Pourquoi y a-t-il une asymétrie ($|T_{min}| \neq |T_{max}|$) pour les signés ?

3 Conversions Signed $\leftrightarrow$ Unsigned

Exercice 8 : Même motif

Pour chaque motif binaire sur 8 bits, donner sa valeur en unsigned ET en signed :

Binaire	Unsigned	Signed
01111111		
10000000		
11111111		
11111110		
11000000		

Exercice 9 : Formules de conversion T2U et U2T

- a) Appliquer $T2U_8(x)$ pour $x = -1$ (sur 8 bits). Vérifier avec la formule.
- b) Appliquer $T2U_8(x)$ pour $x = -128$.

c) Appliquer $U2T_8(u)$ pour $u = 200$.

d) Appliquer $U2T_8(u)$ pour $u = 127$.

Rappel : $T2U_w(x) = x + 2^w$ si $x < 0$, sinon x . $U2T_w(u) = u - 2^w$ si $u > T_{max}$, sinon u .

Exercice 10 : Pièges de comparaison

Prédire le résultat (vrai=1 ou faux=0) de chaque expression C. Justifier.

```
int x = -1;
unsigned int y = 0;

a) x < y // ?
b) x == 0xFFFFFFFF // ? (sur machine 32 bits)
c) -1 < 0U // ?
d) 2147483647 > -2147483648 // ? (types int)
e) 2147483647U > -2147483648 // ? (unsigned à gauche)
```

4 Extension de Signe

Exercice 11 : Extension 8 bits → 16 bits

Effectuer l'extension de signe de 8 vers 16 bits pour :

- a) `int8_t x = -6` (soit 11111010_2) → `int16_t`
- b) `int8_t x = 120` (soit 01111000_2) → `int16_t`
- c) `uint8_t x = 250` (soit 11111010_2) → `uint16_t`

Donner le résultat en binaire (16 bits) et en hexadécimal.

Exercice 12 : Piège de l'extension avec cast

Que valent les variables suivantes ? Expliquer chaque étape.

```
int8_t a = -1; // a = ?
uint32_t b = a; // b = ?
uint32_t c = (uint8_t)a; // c = ?
```

Indice : dans le cas (c), le cast en `uint8_t` est appliqué AVANT l'extension.

5 Dépassements (Overflow)

Exercice 13 : Overflow unsigned

Calculer le résultat des opérations suivantes sur `uint8_t` (8 bits, non signé). Utiliser la règle du modulo 2^8 .

- a) $255 + 1$
- b) $250 + 10$
- c) $0 - 1$
- d) $100 - 150$

Exercice 14 : Overflow signed (Undefined Behavior)

Pour chaque opération sur `int8_t` (8 bits, signé), indiquer :

- Si le résultat est **défini** ou **UB** (Undefined Behavior)
- Le résultat probable (si on suppose un wrap-around, même si c'est UB)

- a) $127 + 1$

- b) $-128 - 1$
- c) $100 + 20$
- d) $-100 - 50$

Exercice 15 : Détection d'overflow AVANT l'opération

Écrire les conditions qui permettent de détecter un overflow **avant** d'effectuer l'opération (pour éviter l'UB).

- a) Pour l'addition `int32_t a + b`, quand y a-t-il overflow positif ?
- b) Pour l'addition `int32_t a + b`, quand y a-t-il overflow négatif ?
- c) Compléter le code :

```
int32_t a = ..., b = ...;
if (/* condition overflow positif */ || /* condition overflow négatif */) {
    printf("Overflow!\n");
} else {
    int32_t sum = a + b; // Safe
}
```

6 IEEE 754 (Flottants)

Exercice 16 : Encodage IEEE 754 float (32 bits)

Encoder les nombres suivants en IEEE 754 float (32 bits). Donner le résultat en hexadécimal.

- a) $+1.0$
- b) -1.0
- c) $+12.5$

Pour chaque nombre, détailler :

- Signe s
- Conversion en binaire fractionnaire
- Normalisation ($1.xxx \times 2^n$)
- Exposant biaisé ($E + 127$)
- Fraction (23 bits, sans le "1" implicite)

Exercice 17 : Décodage IEEE 754

Le float `0xC1480000` est stocké en mémoire. Décoder ce nombre.

- a) Écrire en binaire (32 bits).
- b) Identifier : signe (1 bit), exposant (8 bits), fraction (23 bits).
- c) Calculer l'exposant réel ($E = \text{exposant} - 127$).
- d) Reconstituer la mantisse $M = 1.\text{fraction}$.
- e) Calculer la valeur décimale : $V = (-1)^s \times M \times 2^E$.

Exercice 18 : Valeurs spéciales IEEE 754

Compléter le tableau avec les représentations hexadécimales (32 bits) :

Valeur	s	exp (8 bits)	frac (23 bits)	Hex
+0				
-0				
$+\infty$				
$-\infty$				
NaN (exemple)				

Rappel : $\pm\infty$ a exposant = 255 et fraction = 0. NaN a exposant = 255 et fraction $\neq 0$.

Exercice 19 : Pièges des flottants

- Pourquoi 0.1_{10} n'a-t-il pas de représentation exacte en binaire ?
- Que vaut `0.1f + 0.2f == 0.3f` en C ? Pourquoi ?
- Expliquer pourquoi `(3.14f + 1e10f) - 1e10f` ne donne pas `3.14f`.
- Quelle est la bonne façon de comparer deux flottants ?

Exercice 20 : Analyse de code

Prédire la sortie de chaque `printf`. Justifier.

```

1  #include <stdio.h>
2  #include <math.h>
3
4  int main() {
5      float a = 0.1f + 0.2f;
6      float b = 0.3f;
7      printf("a == b ? %d\n", a == b);
8
9      float x = 1.0f / 0.0f;
10     printf("x = %f, isinf(x) = %d\n", x, isinf(x));
11
12     float y = 0.0f / 0.0f;
13     printf("y == y ? %d, isnan(y) = %d\n", y == y, isnan(y));
14
15     return 0;
16 }
```